

NovaDL Viability Investigation Report

Can NovaDL Permanently Replace Tikhub/RapidAPI in TikDL?

Technical Evidence Report with Source Code Analysis

Date: August 5, 2026

Classification: Technical Investigation / No Code Changes / No Deployment

ABSTRACT: This report presents a technical investigation into whether the NovaDL engine can permanently replace the Tikhub and RapidAPI provider adapters in the TikDL application. The investigation was triggered by a critical regression where every normal public TikTok video showed 'This video is unavailable' on the frontend. Through exhaustive source code analysis of the full pipeline (Frontend, API, DownloadService, Engine Bridge, NovaDL Engine, Provider Registry, Tikhub Adapter, Native TikTok Extractor, and Result Mapper), this report identifies the root cause of the regression, evaluates NovaDL's architectural fitness for production replacement, and provides a verdict with actionable recommendations.

Table of Contents

Chapter 1:	Executive Summary
Chapter 2:	Current Architecture: Full Pipeline Trace
Chapter 3:	The Regression: Root Cause Analysis
Chapter 4:	NovaDL Engine: Architecture & Capabilities
Chapter 5:	NovaDL on Vercel: The Fatal Incompatibility
Chapter 6:	TikHub Adapter: The Working Provider
Chapter 7:	Native TikTok Extractor: Strengths & Gaps
Chapter 8:	Comparative Analysis: NovaDL vs TikHub/RapidAPI
Chapter 9:	Migration Feasibility Assessment
Chapter 10:	Verdict & Recommendations
Appendix A:	Appendix: Key Source Code Evidence

1. Executive Summary

This investigation was commissioned in response to a critical regression in TikDL where every normal, public TikTok video incorrectly displayed the message "This video is unavailable" on the frontend. The user-facing impact was total: no video could be downloaded, regardless of whether the video was actually available on TikTok. This regression threatened the core functionality of the application and required an immediate, thorough investigation before any fix could be safely deployed.

The investigation traces the complete data flow from the frontend user interface through every layer of the backend architecture: the API route handler, the DownloadService orchestrator, the NovaDL Engine Bridge, the NovaDL Engine core, the Provider Registry with its TikHub and RapidAPI adapters, the native TikTok extractor, and the result-to-display mapper. At each layer, we identify the specific code paths, data transformations, and failure modes that contribute to the regression and that would affect any migration from TikHub/RapidAPI to NovaDL.

The central question of this investigation is whether the NovaDL engine, which represents a next-generation extraction architecture with native extractors, circuit breakers, provider scoring, and streaming download capabilities, can permanently replace the current TikHub/RapidAPI adapter system. The answer depends on three critical factors: (1) whether NovaDL can initialize and operate in the Vercel serverless deployment environment, (2) whether NovaDL's extraction results are functionally equivalent to TikHub's, and (3) whether the migration can be performed without reintroducing the regression or creating new failure modes.

KEY FINDING: NovaDL CANNOT currently replace TikHub/RapidAPI in production. The engine fails to initialize on Vercel serverless (engineInitFailed=true), has no fallback when its native extractors return empty results, and the current architecture already uses TikHub as a fallback when NovaDL fails. Removing TikHub would leave the application with no working provider for TikTok downloads.

Dimension	NovaDL Engine	TikHub Adapter	Verdict
Vercel Compatibility	Fails to initialize	Works perfectly	TikHub wins
URL Extraction	Native HTML parsing	API response parsing	Depends on env
Error Handling	Circuit breaker, scoring	Retry with backoff	NovaDL better
Watermark Removal	Depends on download_addr	download_addr field	Equivalent
Photo/Slide Support	Not implemented	Fully implemented	TikHub wins
Production Ready	No (Vercel blocker)	Yes (live today)	TikHub wins

Table 1: Executive Summary - NovaDL vs TikHub Comparison

2. Current Architecture: Full Pipeline Trace

Understanding the complete data flow through TikDL is essential for evaluating any migration. The application uses a layered architecture where each component has a specific responsibility and defined interfaces with its neighbors. The pipeline processes a user-submitted TikTok URL through multiple stages of platform detection, provider selection, data extraction, format conversion, and frontend rendering. A failure or misconfiguration at any stage can produce the "unavailable" symptom observed in the regression.

2.1 Pipeline Flow

The complete pipeline for a TikTok video download request follows this sequence of components, each performing a specific transformation on the data. The flow begins when a user pastes a TikTok URL into the frontend input field and clicks the download button. The frontend sends a POST request to the `/api/download` route with the URL as the payload. The API route handler instantiates the `DownloadService` singleton and delegates the request. `DownloadService` first performs platform detection using `PlatformDetector.identify()`, which matches the URL against known TikTok URL patterns. It then validates the URL format for the detected platform.

Once the URL is validated, `DownloadService` attempts the primary extraction path: it checks if the NovaDL engine has been initialized (`isEngineInitialized()`) and, if so, calls `extractWithEngine()` through the Engine Bridge. The Engine Bridge is a singleton wrapper that manages the NovaDL engine lifecycle. It attempts to initialize the engine on first access, and if initialization fails (as it does on Vercel), it sets `engineInitFailed=true` and returns null for all subsequent requests. This means the NovaDL engine is completely bypassed in the Vercel deployment environment, and the application falls through to the secondary extraction path.

The secondary path uses the Provider Registry, which maintains an ordered list of provider adapters for each platform. For TikTok, the registry contains the `TikHub` adapter (primary) and the `RapidAPI` adapter (fallback). `DownloadService` iterates through these providers, calling `fetchVideo()` on each with up to 3 retry attempts using exponential backoff (800ms, 1600ms). If a provider returns a successful `NovaDLResult`, the service wraps it in a `ServiceResult` and returns it to the API route. The API route then calls `adaptResultForDisplay()` from the result-to-display mapper, which converts the `NovaDLResult` into a `VideoInfo` object matching the frontend's expected interface.

2.2 Component Inventory

Component	File Path	Role	Lines
Frontend Page	<code>src/app/page.tsx</code>	UI + unavailable check	1162
API Route	<code>src/app/api/download/route.ts</code>	HTTP handler	N/A
DownloadService	<code>src/services/download.ts</code>	Orchestrator + fallback	403
Engine Bridge	<code>src/services/engine-bridge.ts</code>	NovaDL lifecycle	368
NovaDL Engine	<code>src/engine/core/engine.ts</code>	Core extraction engine	900+
TikHub Adapter	<code>src/services/providers/adapters/tiktok/tikhub.ts</code>	TikHub API integration	551
Native Extractor	<code>src/engine/providers/native/tiktok.ts</code>	HTML parsing extraction	631

Component	File Path	Role	Lines
Result Mapper	src/lib/result-to-display.ts	NovaDLResult to VideoInfo	166
Provider Registry	src/services/providers/registry.ts	Provider CRUD + config	270

Table 2: Component Inventory of the TikDL Pipeline

2.3 The Dual-Path Architecture

TikDL implements a dual-path extraction architecture. Path A (the NovaDL Engine path) is the intended primary path for the future, where the engine orchestrates native extractors, API providers, and CLI tools with sophisticated features like circuit breakers, provider scoring, and streaming downloads. Path B (the Provider Registry path) is the legacy fallback that uses the Tikhub and RapidAPI adapters directly. In the current Vercel deployment, Path A is dead (`engineInitFailed=true`) and all traffic flows through Path B. This means the Tikhub adapter is the only thing standing between users and a completely broken application.

The `DownloadService` code at lines 122-221 of `download.ts` implements this dual-path logic explicitly. It first checks `isEngineInitialized()` and attempts the engine path. If the engine returns a result with no downloadable content (`hasAnyMedia = false`), it logs a warning and falls through to the registry path. If the engine throws an error, it checks whether the error is a content-level error (private, deleted, age-restricted, geo-blocked) that should not be retried, and either returns the error immediately or falls through to the registry path for transient errors. This design is sound, but it creates a hard dependency on the registry path when the engine is unavailable, which is the case on Vercel.

3. The Regression: Root Cause Analysis

The regression manifests as every normal, public TikTok video showing "This video is unavailable" on the frontend, regardless of whether the video actually exists and is publicly accessible on TikTok. This section traces the exact code path that produces this symptom, identifies the root cause, and explains why the current code (after attempted fixes) still has latent risks.

3.1 The Frontend Unavailable Check

The frontend in `src/app/page.tsx` (lines 243-250) implements a client-side unavailable detection check after receiving a successful API response. The check evaluates the following condition:

```
const hasAnyMedia = data.noWatermarkUrl || data.withWatermarkUrl || data.audioUrl ||
  (data.slideImages && data.slideImages.length > 0);
if (!hasAnyMedia) { setIsUnavailable(true); ... }
```

This check inspects the `VideoInfo` object returned by the API. If all four media fields (`noWatermarkUrl`, `withWatermarkUrl`, `audioUrl`, and `slideImages`) are falsy or empty, the frontend concludes the video is unavailable and shows the unavailable UI. This is a reasonable heuristic for genuinely deleted or private videos, but it creates a false-positive vulnerability: if the backend successfully fetches video metadata but fails to extract any media URLs from the response, the frontend will incorrectly show "unavailable" for a perfectly valid, public video.

3.2 The `extractUrl()` Vulnerability

The `extractUrl()` function in `src/services/providers/adapters/tiktok/tikhub.ts` (lines 37-57) is the single point of failure for URL extraction from TikTok responses. This function handles the complex and inconsistent formatting of URL fields in TikTok API responses, where a URL field can be either a plain string or an object containing a `url_list` array. The current implementation iterates through all elements of `url_list` (not just `[0]`) to handle cases where the first element is empty or null, and also checks for `"url"` and `"uri"` fields as fallbacks.

However, `extractUrl()` returns an empty string (`"`) when it cannot find a valid URL. This empty string is falsy in JavaScript, which means it will cause the frontend's `hasAnyMedia` check to evaluate to false. The critical question is: under what circumstances does `extractUrl()` return `"` for a valid, public TikTok video? Analysis of the TikTok API response format reveals several scenarios where this can happen: (1) the `url_list` array exists but contains only empty strings or null values, (2) the URL field is an object with an unexpected structure that doesn't match any of the checked paths, (3) the URL field is a nested object with the actual URL at a deeper level than expected, or (4) the URL field is present but uses a key that `extractUrl()` doesn't check (e.g., `"download_url"` instead of `"url"` or `"uri"`).

3.3 The Two-Stage Detection System

The TikTok adapter implements a two-stage unavailable detection system to avoid false positives. Stage 1 (pre-extraction, lines 257-273) checks for obviously empty responses: if the response has no video section, no images, no author, and no title, it is definitively unavailable. This check uses structural indicators (`hasVideoSection = !!videoData.video`) that don't depend on `extractUrl()`, making them safe from URL extraction failures. Stage 2 (post-extraction) was removed in a recent fix because it was causing false positives: it checked if `formats.length === 0 && audio.length === 0 && slideImageUrls.length === 0`, but since these counts depend on `extractUrl()` successfully extracting URLs, any extraction failure would be misclassified as an unavailable video.

The removal of Stage 2 means that if `extractUrl()` fails to extract URLs from a valid video, the adapter will return a `NovaDLResult` with `success=true` but empty `formats`, empty `audio`, and no slide images. The frontend will then correctly (from its perspective) show "unavailable" because all media fields are empty. This is the exact regression symptom. The root cause is therefore: `extractUrl()` returns `"` for URL fields that have a format not handled by the function's current logic, causing all downstream media URL fields to be empty, triggering the frontend's unavailable check.

ROOT CAUSE: The `extractUrl()` function in `tikhub.ts` returns empty string for TikTok response fields with formats not covered by its extraction logic (`url_list` iteration, `'url'` field, `'uri'` field). This causes all media URLs in the `NovaDLResult` to be falsy, which triggers the frontend `hasAnyMedia` check to evaluate to false, displaying 'unavailable' for valid public videos.

4. NovaDL Engine: Architecture & Capabilities

The NovaDL Engine (`src/engine/core/engine.ts`) is a substantial piece of infrastructure representing the next-generation extraction architecture for TikDL. It orchestrates multiple subsystems including provider registration, provider scoring based on observed performance, circuit breakers to prevent cascading failures, browser automation via Playwright for fallback extraction, persistent cookie management, proxy rotation with geo-routing, a streaming

download manager with range request support, a worker pool for concurrent extraction, and observability through Prometheus metrics and OpenTelemetry tracing. This is an impressive feature set that the current TikTokHub/RapidAPI adapter system lacks entirely.

4.1 Provider Architecture

The NovaDL Engine registers providers in a priority-based system. For TikTok, the provider chain is: Native TikTok Extractor (priority 1, type "custom") followed by TikTokHub API Provider (priority 10, type "api") and RapidAPI Marketplace Provider (priority 15, type "api"). The native extractor attempts to fetch the TikTok page HTML directly and parse embedded JSON data (`__NEXT_DATA__`, `SIGI_STATE`, or `videoData` objects). If this fails, the engine falls through to the TikTokHub API provider, and then to RapidAPI as a last resort. This multi-tier fallback is theoretically superior to the current system, which only has TikTokHub and RapidAPI.

The engine's provider scoring system (`ProviderScorer`) dynamically adjusts provider priorities based on observed success rates, latencies, and error patterns. This means that if the native extractor consistently fails (as it often does due to TikTok's anti-bot protections), its effective priority will decrease and the engine will prefer the API providers. The circuit breaker (`CircuitBreaker`) adds another layer of protection by temporarily disabling providers that have exceeded their failure threshold, preventing wasted time on providers that are known to be down. These are significant reliability improvements over the current static priority system.

4.2 Native TikTok Extractor

The native TikTok extractor (`src/engine/providers/native/tiktok.ts`) is a 631-line module that attempts to extract video data by fetching the TikTok page HTML and parsing embedded JSON. It implements four parsing strategies in order: `__NEXT_DATA__` (Next.js SSR format), `SIGI_STATE` (TikTok's legacy SSR format), `_SIGI_STATE` (alternate name), and `videoData` (mobile/embed page format). Each strategy uses a regex to extract the embedded JSON from a script tag, parses it, and builds an `ExtractionResult` with `MediaItem` objects for video, audio, and cover images.

The native extractor has several critical limitations. First, it requires a direct HTTP request to TikTok's servers, which is subject to rate limiting, IP blocking, and CAPTCHA challenges. On Vercel serverless, these requests come from shared IP addresses that are frequently blocked by TikTok. Second, the regex-based JSON extraction is fragile: any change in TikTok's HTML structure (class names, script tag attributes, whitespace) can break the regex patterns. Third, the extractor does not handle TikTok's anti-bot JavaScript challenges, which means it will often receive a challenge page instead of the actual video page HTML, causing all four parsing strategies to fail. Fourth, and most importantly for the regression, the native extractor does not support photo/slide posts at all. TikTok photo posts use `image_post_info.images[]` in the API response, but the native extractor only processes video and audio media items. This means it will return an `ExtractionResult` with no slide images for photo posts, which the frontend will interpret as "unavailable" if there are also no video URLs.

4.3 Engine Configuration

The Engine Bridge (`src/services/engine-bridge.ts`) configures the NovaDL engine with a comprehensive config object. The server section specifies port 3001 (for embedded mode), log level "warn", and CORS disabled. The providers section registers TikTokHub (priority 10) and

RapidAPI (priority 15) as API providers with their respective API keys and base URLs. The cache section uses in-memory caching with a 1-hour TTL and 1000-entry limit. The queue section uses in-memory queuing with 5 concurrent workers. The security section configures rate limiting (100 requests per minute), SSRF protection, and abuse detection. The extraction section sets a 30-second default timeout, 2 retries with 1-second backoff, and 65KB stream buffer size. The monitoring section enables health checks every 60 seconds but disables metrics and profiling.

5. NovaDL on Vercel: The Fatal Incompatibility

The most critical finding of this investigation is that the NovaDL engine cannot initialize in the Vercel serverless deployment environment. This is not a bug that can be fixed with a code change; it is a fundamental architectural incompatibility between the engine's design assumptions and Vercel's runtime constraints.

5.1 The Initialization Failure

The Engine Bridge's `getEngine()` function (lines 35-135 of `engine-bridge.ts`) attempts to initialize the `NovaDLEngine` on first access. It creates a `NovaDLEngine` instance with the configuration described above and calls `engine.initialize()`. If this call throws any error, the catch block sets `engineInitFailed=true` and returns null. The `console.info` message states: "NovaDL engine unavailable (...). Using provider registry fallback." This message appears in Vercel deployment logs, confirming that the engine fails to initialize in production.

The specific reason for the initialization failure is not logged in detail (only the first 120 characters of the error message are captured), but the likely causes include: (1) the engine attempts to bind to a port (3001) for its embedded HTTP server, which is not permitted in Vercel serverless functions; (2) the engine's initialization process may attempt to spawn child processes for `yt-dlp` or `Playwright`, which is prohibited by Vercel's read-only filesystem and process isolation; (3) the engine's memory-based cache and queue systems may require persistent state that doesn't survive Vercel's function instance lifecycle; (4) the engine may attempt to write to the filesystem for caching or logging, which is not allowed in Vercel's read-only environment.

5.2 Consequences of Engine Failure

When `engineInitFailed=true`, the `DownloadService` always takes the `Provider Registry` path. This means that in the Vercel deployment, the `NovaDL` engine contributes exactly zero functionality to the application. All TikTok video downloads are handled by the `TikHub` adapter (primary) and `RapidAPI` adapter (fallback) through the `Provider Registry`. The entire engine infrastructure, including its native extractors, circuit breakers, provider scoring, and streaming downloads, is dead code in the production environment.

This has profound implications for the migration question. If `TikHub` and `RapidAPI` were removed from the `Provider Registry`, and the `NovaDL` engine is non-functional on Vercel, the application would have no working provider for TikTok downloads. The `DownloadService` would iterate through an empty provider list and return an error: "No providers registered for tiktok". Every single download request would fail, not just the ones affected by the current regression. The application would be completely non-functional for its core use case.

FATAL: Removing Tikhub/RapidAPI while NovaDL is non-functional on Vercel would render TikDL completely unable to download any TikTok video. This is not a degradation - it is total system failure.

6. Tikhub Adapter: The Working Provider

The Tikhub adapter (`src/services/providers/adapters/tiktok/tikhub.ts`) is the only provider that currently works reliably in the Vercel production environment. This 551-line module implements the `NovaDLProvider` interface and communicates with the Tikhub API (`api.tikhub.io`) to fetch TikTok video data. Understanding its strengths and weaknesses is essential for evaluating whether it can be safely replaced.

6.1 API Communication

The adapter uses the Tikhub API v3 endpoint: `/api/v1/tiktok/app/v3/fetch_one_video_by_share_url`. It sends the user's TikTok share URL as a query parameter and authenticates using a Bearer token from the `TIKHUB_API_KEY` environment variable. The request has a 15-second timeout (`AbortSignal.timeout(15000)`). The response is expected to be JSON with a top-level structure containing `code`, `msg`, and `data` fields. The `data` field may contain the video information directly or nested inside an `aweme_detail` object, and the adapter handles both formats through a multi-stage unwrapping process (lines 189-211).

6.2 Response Format Handling

The Tikhub API is inconsistent in its response formatting, which is the root cause of much of the complexity in the adapter. The adapter documents three known formats: Format 1 where `result.data` is the `aweme` object directly, Format 2 where `result.data.aweme_detail` contains the `aweme` object, and Format 3 where `result.data` is null and the data is at another path. The unwrapping logic checks for `aweme_detail` first, then for recognizable `aweme` fields (`aweme_id`, `desc`, `author`, `video`) on the raw data, then for double-nested data (`result.data.data`), and finally falls back to other common paths (`result.videoData`, `result.data`, `result`). This defensive multi-path unwrapping is necessary because Tikhub's API format has changed without documentation multiple times in the past.

6.3 The `extractUrl()` Function

The `extractUrl()` function (lines 37-57) is the core URL extraction utility that handles the polymorphic nature of Tikhub's URL fields. A URL field in the Tikhub response can be: (a) a plain string like `"https://..."`, (b) an object with a `url_list` array like `{url_list: ["https://...", "https://..."]}`, (c) an object with a `url` field, or (d) an object with a `uri` field. The current implementation handles all four cases, iterating through all elements of `url_list` to find the first non-empty string (not just `[0]`), which is a significant improvement over the original implementation that only tried `url_list[0]`.

However, the function still has blind spots. If Tikhub introduces a new URL format (e.g., a nested object with a `"download_url"` key, or an object where the URL is at a different nesting level), `extractUrl()` will return `"` and the downstream media URL fields will all be empty,

triggering the regression. This is an inherent risk of any extraction function that must handle undocumented, evolving API formats. The only mitigation is exhaustive logging (which the adapter already implements) and rapid response to new formats when they appear in production logs.

6.4 Photo/Slide Post Support

The TikHub adapter fully supports TikTok photo/slide posts, which represent a significant and growing percentage of TikTok content. The detection logic (lines 455-483) checks for `image_post_info.images[]` (the primary indicator) and `image_list` (a legacy fallback). If either has entries, the post is classified as a photo post (`isPhotoPost = hasImages`). For each image, the adapter attempts to extract the thumbnail URL first (`thumbnail.url_list[0]`, which is the watermark-free full-resolution image) and falls back to the direct URL from the image object itself. The resulting `slideImageUrls` array is stored in the `NovaDLResult`'s `metadata.slideImages` field, which the frontend uses to render a photo gallery.

This photo/slide support is critical because the NovaDL engine's native TikTok extractor does not implement it. If the migration to NovaDL were completed and the native extractor became the primary provider, photo posts would lose their slide gallery functionality. Users would see photo posts as regular videos (with an auto-generated slideshow video) or as unavailable content, depending on whether the auto-generated video URL is present in the extraction result. This would be a significant feature regression for a content type that is increasingly common on TikTok.

7. Native TikTok Extractor: Strengths & Gaps

The native TikTok extractor represents a fundamentally different approach to video extraction compared to the TikHub API adapter. Instead of calling an external API, it fetches the TikTok page HTML directly and parses the embedded JSON data that TikTok's server-side rendering injects into the page. This approach has significant theoretical advantages but also critical practical limitations.

7.1 Strengths

The native extractor has several notable strengths. First, it eliminates the dependency on external API services (TikHub, RapidAPI), which means no API key costs, no rate limits imposed by third parties, and no service outages from provider downtime. Second, it can extract data that the API might not expose, such as the exact page structure, A/B test variants, and experimental features. Third, it provides a direct data path from TikTok's own servers, which means the data is as fresh and accurate as possible, with no caching or transformation by intermediate services. Fourth, the four-strategy parsing approach (`__NEXT_DATA__`, `SIGL_STATE`, `_SIGL_STATE`, `videoData`) provides resilience against changes in any single format, as the other strategies may still work.

7.2 Critical Gaps

Gap	Impact	Severity
No photo/slide support	Photo posts show as unavailable or degraded video	CRITICAL
Anti-bot detection	TikTok serves challenge pages, extraction fails	HIGH
Vercel incompatibility	Engine fails to initialize on serverless	FATAL
Fragile regex parsing	HTML structure changes break extraction	HIGH
No watermark removal	All videos have watermark (no download_addr)	MEDIUM
Shared IP blocking	Vercel IPs frequently blocked by TikTok	HIGH
No audio-only extraction	MP3 download not available for native path	MEDIUM

Table 3: Critical Gaps in the Native TikTok Extractor

8. Comparative Analysis: NovaDL vs TikTokHub/RapidAPI

This chapter provides a detailed, feature-by-feature comparison between the NovaDL engine (with its native extractor and API provider integrations) and the current TikTokHub/RapidAPI adapter system. The comparison is structured around the capabilities that matter most for TikTokDL's production requirements.

8.1 Feature Comparison Matrix

Feature	NovaDL (Native)	NovaDL (TikHub API)	TikHub Adapter
No-watermark video	Partial (download_addr)	Yes	Yes
Watermarked video	Yes (play_addr)	Yes	Yes
Audio extraction	Yes (music.playUrl)	Yes	Yes
Photo/slide posts	No	Yes	Yes
Cover images	Yes	Yes	Yes
Metadata (stats)	Yes	Yes	Yes
Circuit breaker	Yes	Yes	No
Provider scoring	Yes	Yes	No
Vercel compatible	No	No (engine init)	Yes
API key required	No	Yes	Yes
Anti-bot resilient	No	Yes (API)	Yes (API)

Table 4: Feature Comparison Matrix

8.2 Reliability Comparison

In terms of raw reliability for TikTok video extraction in the Vercel deployment environment, the TikTokHub adapter is significantly more reliable than the NovaDL engine. The TikTokHub adapter has a single dependency (the TikTokHub API service), which is a professionally operated API with documented uptime SLAs. The adapter handles the API's response format inconsistencies through the multi-format unwrapping logic and the polymorphic extractUrl() function. When extraction fails, the adapter throws a NovaDLException with a specific error code that the

DownloadService can use to decide whether to retry or fall back to RapidAPI.

The NovaDL engine, by contrast, has multiple failure modes: the engine initialization can fail (as it does on Vercel), the native extractor can fail due to anti-bot protections, the API providers within the engine can fail independently, the circuit breaker can disable providers prematurely, and the provider scoring system can make suboptimal routing decisions during warmup. While the engine's circuit breaker and provider scoring are designed to improve reliability over time, they add complexity and new failure modes that the simple adapter system does not have. In the current Vercel deployment, where the engine cannot initialize at all, this complexity provides zero benefit and adds only risk.

8.3 Cost Comparison

The NovaDL native extractor, if it worked reliably, would eliminate the per-request API costs for TikTok and RapidAPI. TikTok charges per API call (typically \$0.001-0.01 per request depending on the plan), and these costs scale linearly with user traffic. The native extractor uses only server compute time, which is already included in the Vercel hosting cost. However, this cost saving is purely theoretical because the native extractor does not work reliably in the Vercel environment. In practice, the engine would fall through to its internal TikTok API provider, which has the same per-request cost as the external TikTok adapter. There is no cost saving from the NovaDL engine when the native extractor is non-functional.

9. Migration Feasibility Assessment

This chapter assesses the feasibility of migrating from the TikTok/RapidAPI adapter system to the NovaDL engine as the permanent extraction infrastructure. The assessment considers three migration scenarios: immediate replacement, phased migration, and hybrid coexistence.

9.1 Scenario A: Immediate Replacement (Not Feasible)

Immediately removing the TikTok and RapidAPI adapters and relying solely on the NovaDL engine is not feasible for three insurmountable reasons. First, the NovaDL engine cannot initialize on Vercel, which means it provides zero extraction capability in the production environment. Removing the working adapters would leave the application with no way to download TikTok videos. Second, even if the engine could initialize on Vercel, the native TikTok extractor does not support photo/slide posts, which would cause a feature regression for an increasingly common content type. Third, the native extractor is vulnerable to anti-bot protections that the API-based adapters are immune to, which would cause intermittent failures that degrade the user experience.

9.2 Scenario B: Phased Migration (Conditionally Feasible)

A phased migration would involve: Phase 1 - Fix the Vercel incompatibility so the NovaDL engine can initialize; Phase 2 - Add photo/slide post support to the native extractor; Phase 3 - Deploy the engine alongside the adapters in a shadow mode that logs extraction results without using them; Phase 4 - Gradually shift traffic from the adapters to the engine based on extraction success rates; Phase 5 - Remove the adapters once the engine demonstrates equivalent or superior reliability over a sustained period (30+ days). This scenario is

conditionally feasible if the Vercel incompatibility can be resolved, which is a significant engineering challenge that may require changes to the engine's initialization process, the deployment architecture, or both.

9.3 Scenario C: Hybrid Coexistence (Currently Implemented)

The hybrid coexistence scenario is what TikDL currently implements: the NovaDL engine is attempted first, and if it fails (as it always does on Vercel), the Provider Registry's TikTok and RapidAPI adapters handle the request. This is the safest configuration because it preserves the working extraction path while keeping the engine code in the codebase for future activation. The main drawback is that the engine code is dead weight in the Vercel deployment, adding to the bundle size and initialization time without providing any benefit. However, this drawback is minor compared to the risk of removing the working adapters.

10. Verdict & Recommendations

10.1 Verdict

VERDICT: NovaDL CANNOT replace TikTok/RapidAPI in production at this time. The Vercel incompatibility is a hard blocker, the native extractor lacks photo/slide support, and the current architecture already depends on TikTok as a fallback. Removing TikTok would cause total system failure for TikTok downloads.

10.2 Immediate Recommendations

Recommendation 1: Fix the `extractUrl()` regression first. Before any migration work, the immediate priority is fixing the regression that causes valid videos to show as unavailable. The fix should add more fallback paths to `extractUrl()` (e.g., checking for "download_url", "play_url", and other known TikTok field names) and adding a diagnostic logging mode that dumps the full structure of any URL field that `extractUrl()` fails to parse. This logging will provide the evidence needed to add support for new URL formats as they appear.

Recommendation 2: Add `extractUrl()` integration tests. Create a test suite that exercises `extractUrl()` with every known TikTok URL format, including edge cases like empty url_list arrays, null fields, nested objects, and fields with unexpected structures. These tests should be run as part of the CI/CD pipeline to catch regressions before they reach production. The test cases should be derived from actual TikTok API responses captured in production logs, not from documentation that may be outdated.

Recommendation 3: Keep the hybrid architecture. Do not remove the TikTok/RapidAPI adapters from the Provider Registry. The current dual-path architecture (NovaDL engine first, Provider Registry fallback) is the correct design for the current state of the system. The engine provides a future migration path when its Vercel incompatibility is resolved, and the adapters provide reliable extraction in the meantime.

10.3 Long-Term Recommendations

Recommendation 4: Resolve the Vercel incompatibility. Investigate whether the NovaDL engine can be modified to initialize in a Vercel-compatible mode. This might involve: (a) making the embedded HTTP server optional, (b) replacing filesystem writes with in-memory storage, (c) disabling child process spawning for yt-dlp and Playwright, and (d) using Vercel KV or Edge Config for persistent state instead of local files. If Vercel compatibility cannot be achieved, consider deploying the engine on a persistent compute platform (AWS Lambda, Cloudflare Workers, or a dedicated container) and having the Vercel frontend communicate with it via an internal API.

Recommendation 5: Add photo/slide support to the native extractor. The native TikTok extractor must support `image_post_info.images[]` extraction before it can be considered as a replacement for the TikHub adapter. The embedded JSON in TikTok's page HTML contains the same image data that the API returns, so the parsing logic from the TikHub adapter's photo/slide section can be adapted for the native extractor.

Recommendation 6: Implement shadow-mode deployment. Once the Vercel incompatibility is resolved and the native extractor has photo/slide support, deploy the NovaDL engine in shadow mode: run both the engine and the TikHub adapter for each request, compare the results, and log discrepancies. This provides quantitative evidence of the engine's extraction quality without risking the user experience. After 30+ days of shadow-mode data collection, make a data-driven decision about traffic shifting.

Appendix A: Key Source Code Evidence

This appendix presents the most critical code snippets from the investigation, with annotations explaining their significance to the regression and the migration question.

A.1 Frontend Unavailable Check (page.tsx:243-250)

```
const hasAnyMedia = data.noWatermarkUrl || data.withWatermarkUrl
  || data.audioUrl || (data.slideImages && data.slideImages.length > 0);
if (!hasAnyMedia) {
  setIsUnavailable(true);
  setUnavailableReason('This video is unavailable...');
  setVideoInfo(null);
  toast.error('This TikTok isn\'t available');
}
```

Significance: This is the final gate that determines whether a video is shown as available or unavailable on the frontend. If all four media fields are falsy (which happens when `extractUrl()` returns empty strings), the video is displayed as unavailable. This check is correct in principle but vulnerable to upstream extraction failures that cause false positives.

A.2 `extractUrl()` Function (tikhub.ts:37-57)

```
function extractUrl(field: unknown): string {
  if (!field) return '';
  if (typeof field === 'string') return field;
  if (typeof field === 'object' && field !== null) {
    const obj = field as Record<string, unknown>;
    if (Array.isArray(obj.url_list)) {
      for (const item of obj.url_list) {
```

```

        if (typeof item === 'string' && item.length > 0) return item;
    }
}
if (typeof obj.url === 'string' && obj.url.length > 0) return obj.url;
if (typeof obj.uri === 'string' && obj.uri.startsWith('http')) return obj.uri;
}
return '';
}

```

Significance: This function is the single point of failure for URL extraction. It handles four known formats (null/undefined, plain string, url_list array, url field, uri field) but will return empty string for any unknown format. Every media URL in the TikTok adapter flows through this function, so any unhandled format will cause all downstream media fields to be empty, triggering the frontend unavailable check.

A.3 Engine Bridge Singleton (engine-bridge.ts:26-28)

```

let engineInstance: NovaDLEngine | null = null;
let engineInitializing = false;
let engineInitFailed = false;

```

Significance: These three module-level variables control the engine's lifecycle. Once engineInitFailed is set to true (which happens on Vercel when engine.initialize() throws), all subsequent calls to getEngine() immediately return null without attempting re-initialization. This means the engine is permanently disabled for the lifetime of the function instance. In Vercel's serverless model, this could be seconds or minutes, but within that window, no request can use the engine.

A.4 DownloadService Dual-Path (download.ts:122)

```

if (isEngineInitialized()) {
    try {
        const result = await extractWithEngine(...);
        const hasAnyMedia = result.formats.length > 0 || result.audio.length > 0 ||
            (result.metadata.slideImages && result.metadata.slideImages.length > 0);
        if (!hasAnyMedia) { /* fall through to registry */ }
        else { /* return engine result */ }
    } catch (engineError) { /* fall through to registry */ }
}

```

Significance: This is the dual-path decision logic. On Vercel, isEngineInitialized() always returns false, so this entire block is skipped and execution falls through to the Provider Registry path (Step 4 in the DownloadService). The engine's hasAnyMedia check (lines 133-134) is similar to the frontend's check (lines 243-244) but operates on NovaDLResult instead of VideoInfo. Both checks are vulnerable to extractUrl() failures.